

**Μοντελοποίηση και προγραμματισμός
περιορισμών**

Πρόβλημα ωρολογίου προγράμματος



AM: 161173

Ονοματεπώνυμο: Βεσέλ Νταλίπαϊ

ΜΕΡΟΣ Α

Στον κώδικα ορίζονται τα σύνολα φοιτητών S , μαθημάτων L , αιθουσών R , διδασκόντων $teachers$ και χρονικών διαστημάτων $timeslots/T$.

Τα δεδομένα του προβλήματος δίνονται μέσω λεξικών όπως $student_courses$, $room_capacities$, $course_teachers$ και $teacher_availability$, ενώ οι χρονικές εξαρτήσεις εκφράζονται με τη λίστα $precedence$. Το dataset είναι 5 φοιτητές, 7 μαθήματα, 3 αίθουσες και 6 χρονικές στιγμές.

Χρησιμοποιούνται δυαδικές μεταβλητές $x[(c,t)]$ για την ανάθεση μαθήματος c σε χρονική στιγμή t και $y[(c,r)]$ για την ανάθεση μαθήματος c σε αίθουσα r .

Επιπλέον ορίζεται η $z[(c,t,r)]$ για τον συνδυασμό μάθημα, ώρα, αίθουσα, η οποία γίνεται 1 μόνο όταν το μάθημα c έχει ανατεθεί ταυτόχρονα στη χρονική στιγμή t και στην αίθουσα r . Αυτό εξασφαλίζεται με τους περιορισμούς σύνδεσης που υπάρχουν στον κώδικα, ώστε η z να ενεργοποιείται μόνο όταν ισχύουν και οι δύο αντίστοιχες αναθέσεις.

Για κάθε μάθημα επιβάλλεται να ανατεθεί ακριβώς μία χρονική στιγμή και ακριβώς μία αίθουσα.

Η χωρητικότητα υλοποιείται υπολογίζοντας το πλήθος φοιτητών ανά μάθημα ($course_size$) και απενεργοποιώντας αίθουσες που δεν επαρκούν για το συγκεκριμένο μάθημα.

Για την αποφυγή επικαλύψεων, κάθε φοιτητής μπορεί να παρακολουθεί το πολύ ένα από τα μαθήματά του σε κάθε χρονική στιγμή, και κάθε αίθουσα μπορεί να φιλοξενεί το πολύ ένα μάθημα ανά ώρα.

Η διαθεσιμότητα διδασκόντων υλοποιείται απενεργοποιώντας $timeslots$ όπου ο διδάσκων δεν είναι διαθέσιμος, ενώ επιπλέον εξασφαλίζεται ότι κάθε διδάσκων διδάσκει το πολύ ένα μάθημα την ίδια ώρα.

Τέλος, οι χρονικές εξαρτήσεις υλοποιούνται με βοηθητικές ακέραιες μεταβλητές $time_of$ που αντιστοιχούν το $timeslot$ κάθε μαθήματος με τις δυαδικές μεταβλητές x . Για κάθε ζεύγος στη $precedence$ επιβάλλεται το πρώτο μάθημα να προγραμματιστεί πριν από το δεύτερο, εξασφαλίζοντας τη σειρά διάλεξης πριν εργαστηρίου.

ΜΕΡΟΣ Β

1.Χρονικές εξαρτήσεις

Οι χρονικές εξαρτήσεις κάνουν το πρόβλημα πιο δύσκολο γιατί μειώνουν τις διαθέσιμες λύσεις και συνδέουν μαθήματα που αλλιώς θα μπορούσαν να τοποθετηθούν ανεξάρτητα, αυξάνοντας τις πιθανές συγκρούσεις με φοιτητές/καθηγητές/αίθουσες. Στο πρόγραμμα οι περιορισμοί τροποποιούνται δημιουργώντας για κάθε μάθημα μια ακέραια μεταβλητή `time_of[c]` που παίρνει το `timeslot` του μαθήματος από τις δυαδικές μεταβλητές `x[(c,t)]`, και μετά επιβάλλουμε τη σειρά με τον περιορισμό `model.Add(time_of[lecture] < time_of[lab])`

2.Hard vs Soft Constraints

Soft constraints είναι όσα δεν είναι απαραίτητα για να είναι έγκυρο το πρόγραμμα, αλλά βελτιώνουν την ποιότητα της λύσης. Π.χ. να αποφεύγονται κενά στο ωράριο ενός φοιτητή ή να προτιμώνται συγκεκριμένα `timeslots` για έναν διδάσκοντα αντί απλά να είναι διαθέσιμος. Για να τα ενσωματώναμε με ποινές στο CP-SAT, θα προσθέταμε για κάθε ανεπιθύμητη κατάσταση μια δυαδική μεταβλητή `penalty` που γίνεται 1 όταν συμβαίνει, θα τη συνδέαμε με τους αντίστοιχους περιορισμούς και μετά θα ορίζαμε στόχο ελαχιστοποίησης με `model.Minimize(...)`, π.χ. ένα άθροισμα σταθμισμένων ποινών `weight * penalty`, ώστε ο solver να βρίσκει εφικτό πρόγραμμα που ταυτόχρονα μειώνει όσο γίνεται τις παραβιάσεις των soft constraints.

3.Κενά και ομογενοποίηση προγράμματος

Για να ελαχιστοποιήσουμε τα κενά και να ομογενοποιήσουμε το πρόγραμμα, θα το χειριζόμασταν ως soft constraint με ποινές. Για κάθε φοιτητή θα μετράμε πόσο απλωμένα είναι τα μαθήματά του μέσα στη μέρα και θα τιμωρούμε περιπτώσεις όπου εμφανίζεται μοτίβο μάθημα–κενό–

μάθημα. Πρακτικά, ορίζουμε δυαδικές μεταβλητές `penalty` που ενεργοποιούνται όταν υπάρχουν κενά ανάμεσα σε δύο ενεργά slots ή όταν το πρώτο και το τελευταίο μάθημα της μέρας απέχουν πολύ και τις συνδέουμε με τις υπάρχουσες μεταβλητές $x[(c,t)]$ ώστε να εντοπίζονται αυτές οι καταστάσεις. Τέλος, ορίζουμε στόχο ελαχιστοποίησης με `model.Minimize(...)`, ώστε ο solver να προτιμά λύσεις με πιο συμπαγές πρόγραμμα χωρίς να παραβιάζει τα `hard constraints`.

4.Επιλογή τεχνικής επίλυσης

Θα επέλεγα heuristics μέσα σε CP-SAT και όχι καθαρή εξαντλητική αναζήτηση ή meta-heuristics. Η εξαντλητική αναζήτηση δεν κλιμακώνει πρακτικά, γιατί ο χώρος λύσεων μεγαλώνει εκθετικά όσο αυξάνονται μαθήματα/αίθουσες/slots. Οι meta-heuristics μπορούν να δώσουν καλές λύσεις, αλλά θέλουν αρκετό tuning, δεν εγγυώνται εύκολα εφικτότητα σε σκληρούς περιορισμούς και είναι πιο δύσκολες να παρουσιαστούν/ελεγχθούν. Αντίθετα, το CP-SAT που χρησιμοποιούμε ήδη χειρίζεται φυσικά τα `hard constraints`, βρίσκει `feasible/optimal` λύσεις με ισχυρούς ενσωματωμένους μηχανισμούς αναζήτησης και επιτρέπει καθαρά και σωστά να προσθέσουμε και `soft constraints` μέσω `model.Minimize(...)` με ποινές, άρα είναι πιο αξιόπιστη και άμεσα εφαρμόσιμη επιλογή για το συγκεκριμένο πρόβλημα.

Σχολιασμός των αποτελεσμάτων

Από την εκτύπωση του ωρολογίου προγράμματος φαίνεται ότι όλα τα μαθήματα έχουν τοποθετηθεί ακριβώς μία φορά σε κάποιο timeslot και σε μία αίθουσα, χωρίς να εμφανίζονται διπλοκρατήσεις της ίδιας αίθουσας στην ίδια χρονική στιγμή και σε κάθε ώρα υπάρχει το πολύ ένα μάθημα ανά αίθουσα. Επίσης, η χρονική εξάρτηση τηρείται: το `Course1_Lec` έχει προγραμματιστεί τη Δευτέρα 10:00 ενώ το `Course1_Lab` την Τρίτη 09:00, άρα το εργαστήριο μπαίνει μετά τη διάλεξη. Παρατηρείται ότι η λύση χρησιμοποιεί κυρίως την `Room3` και την `Room2`, ενώ αρκετές αίθουσες/ώρες μένουν κενές. Αυτό είναι αναμενόμενο γιατί στο μοντέλο

υπάρχουν μόνο hard constraints, οπότε ο solver αρκείται να βρει μια εφικτή ανάθεση που σέβεται τους περιορισμούς.